

Reviewer Pack — Treewidth of Prime-Implicant Compatibility Graph Tracks DNF-...

Entry 614f26dd51ba · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 23:14:14 UTC

Treewidth of Prime-Implicant Compatibility Graph Tracks DNF-MCSP

Entry ID: 614f26dd51ba **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 23:14:14 UTC

1. Conjecture

Field A (mathematical branch): Robertson–Seymour graph minor theory — graph treewidth $\text{tw}(G)$ defined via optimal tree decompositions of bag-width $\leq k+1$, accessed by the equivalent vertex-elimination characterization $\text{tw}(G) = \min \text{ over orderings } \pi \text{ of } \max_v |N^>(v)|$ in the fill-in graph (Bodlaender 1996; Robertson–Seymour Graph Minors V; Arnborg–Corneil–Proskurowski 1987). A non-ring, set-comparison/min-over-orderings functional — treewidth is preserved under graph minors, NOT under polynomial ring lifts (treewidth of the fill-in pattern has no F_q polynomial surrogate), so the bridge is Aaronson–Wigderson algebrization-safe. <5 papers linking treewidth of prime-implicant intersection graphs to MCSP / minimum DNF size (extant treewidth work is on the input CNF’s primal/dual/incidence graphs for SAT parameterized complexity à la Samer–Szeider, never on the PI compatibility graph as a meta-complexity invariant). Distinct from blacklisted FCA antichain width (a Galois-lattice width on the implicant context, not a graph-minor invariant), Mostar / Forman–Ricci / fatgraph / dismantlability (none uses tree-decomposition / elimination-ordering width), and the Mahler-measure single-number truth-table trap. **Field B** (complexity object): DNF-MCSP — the minimum-DNF leaf size $\sigma_{\text{DNF}}(f)$ of $f: \{0,1\}^n \rightarrow \{0,1\}$, in the Masek 1979 / Umans 1999 / Allender–Hellerstein–McCabe–Pitassi–Saks / Hirahara meta-complexity regime targeting MCSP-style NP-hardness and crypto reductions; the canonical Quine–McCluskey set-cover formulation where σ_{DNF} is the optimum of a prime-implicant cover of minterms. The invariant $\text{tw}(G(f))$ is RELATIONAL (a graph parameter of the PI compatibility graph), not a single truth-table number, sidestepping the trap (j) that killed Mahler-measure.

Statement:

For every Boolean $f: \{0,1\}^n \rightarrow \{0,1\}$ with at least two prime implicants, let $G(f)$ be the prime-implicant compatibility graph: vertices = $\text{PI}(f)$ (computed by Quine–McCluskey), edges $\{p,q\}$ iff the partial assignments p and q are mutually consistent (no variable assigned to opposing values). Let $\tau(f) := \sigma_{\text{DNF}}(f)$ measure the minimum number of TERMS in any DNF for f . Then $\tau(f) \geq \text{tw}(G(f)) + 1$, where tw is graph treewidth; equivalently, every DNF representation of f has at least $\omega^*(G(f)) := \text{tw}(G(f)) + 1$ terms. A single instance with $\tau(f) \leq \text{tw}(G(f))$ falsifies.

Rationale (proposer’s reasoning):

Two prime implicants that are mutually consistent share a non-empty extension subcube, hinting that any minimum cover must ‘separate’ them by at least one term — the bag-overlap of an optimal tree decomposition of $G(f)$ is the natural combinatorial obstruction to

this separation. The Robertson–Seymour width parameters have been wildly successful in parameterized complexity (Courcelle’s theorem, MSO-definability) yet completely absent from MCSP / minimum-DNF lower bounds, even though the prime-implicant cover problem is exactly the kind of monotone clutter problem where bag-overlap should obstruct cover size. The invariant is structural on $PI(f)$ rather than on the truth table directly, and treewidth is comparison/set-based, dodging both algebrization and the truth-table single-number natural-proofs trap.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c235aaf9f2485b09

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 90 instances ($n \in \{6, 8, 10\}$, 30 seeds each), conjecture is SUPPORTED iff every instance satisfies $\sigma_DNF(f) \geq tw(G(f)) + 1$. FALSIFIED if any single instance has $\sigma_DNF(f) < tw(G(f)) + 1$. Aggregate: report $Pearson(\sigma_DNF, tw+1)$ and worst-case ratio $\sigma_DNF/(tw+1) \geq 1.0$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.86	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - treewidth prime implicant compatibility graph DNF minimization - tree decomposition minimum DNF size meta-complexity MCSP - elimination ordering prime implicant graph Boolean function complexity lower bound

Top relevant hits considered: - [<http://arxiv.org/abs/1408.6321v1>] Crossing Minimization for 1-page and 2-page Drawings of Graphs with Bounded Treewidth - [<http://arxiv.org/abs/1210.8205v2>] Treewidth of the Line Graph of Complete and Complete Multipartite Graphs - [<http://arxiv.org/abs/0909.3221v2>] The Stackelberg Minimum Spanning Tree Game on Planar and Bounded-Treewidth Graphs - [<http://arxiv.org/abs/1805.08177v4>] A Polynomial Time Delta-Decomposition Algorithm for Positive DNFs - [<http://arxiv.org/abs/1204.1444v2>] Computing the minimum rank of a loop directed tree - [<http://arxiv.org/abs/2412.05161v1>] DNF: Unconditional 4D Generation with Dictionary-based Neural Fields - [<http://arxiv.org/abs/2511.07739v3>] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [<http://arxiv.org/abs/2307.07605v1>] First-order Methods for Affinely Constrained Composite Non-convex Non-smooth Problems: Lower Complexity Bound and Near-o

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1,

elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def quine_mccluskey(f):
        n = len(f[0])
        minterms = [i for i in range(2**n) if f[i]]
        pi = []

        while minterms:
            next_pi = set()
            singletons = {}

            for term in minterms:
                for i in range(n):
                    mask = 1 << i
                    neighbor = term ^ mask
                    if neighbor in minterms:
                        if (term, neighbor) not in singletons:
                            singletons[(term, neighbor)] = []
                        singletons[(term, neighbor)].append(term)

            for pair, terms in singletons.items():
                next_pi.add(pair[0])
                for t in terms:
                    minterms.remove(t)

            pi.extend(next_pi)

        return pi

    def is_compatible(p, q):
        for i in range(len(p)):
            if p[i] != 'x' and q[i] != 'x' and p[i] != q[i]:
                return False
        return True

    def build_graph(pi):
        n = len(pi)
        adj = [[0]*n for _ in range(n)]

        for i in range(n):
            for j in range(i+1, n):
                if is_compatible(pi[i], pi[j]):
                    adj[i][j] = 1
                    adj[j][i] = 1

        return adj

    def treewidth(adj):
```

```

n = len(adj)
if n == 0:
    return -1

def dfs(node, parent, bag):
    nonlocal max_bag_size
    max_bag_size = max(max_bag_size, len(bag))

    for neighbor in range(n):
        if adj[node][neighbor] and neighbor != parent:
            new_bag = set(bag)
            new_bag.add(neighbor)
            dfs(neighbor, node, new_bag)

max_bag_size = 0
dfs(0, -1, {0})
return max_bag_size - 1

def sigma_dnf(pi):
    n = len(pi)
    m = len(f)
    cover_matrix = [[0]*m for _ in range(n)]

    for i in range(n):
        for j in range(m):
            if all(pi[i][k] == 'x' or pi[i][k] == f[j][k] for k in range(n)):
                cover_matrix[i][j] = 1

    def branch_and_bound(row, col, count):
        if row == n:
            return count
        min_count = float('inf')

        for j in range(col, m):
            if cover_matrix[row][j]:
                new_cover_matrix = [row[:] for row in cover_matrix]
                for i in range(n):
                    if pi[i][row] != 'x' and pi[i][row] == f[j][i]:
                        new_cover_matrix[i][j] = 0
                min_count = min(min_count, branch_and_bound(row+1, j+1, count + (new_cover_mat

        return min_count

    return branch_and_bound(0, 0, 0)

n = random.choice([6, 8, 10])
k = 4
terms = [random.sample(range(n), 3) for _ in range(k)]
f = [[any(pi[i][j] == 'x' or pi[i][j] == term[j] for j in range(n)) for i in range(2**n)] for

pi = quine_mccluskey(f)
adj = build_graph(pi)
tw = treewidth(adj)
sigma_dnf_val = sigma_dnf(pi)

return {

```

```

        "metric_name": "sigma_dnf",
        "metric_value": sigma_dnf_val,
        "instances_tested": 1,
        "conjecture_holds": sigma_dnf_val >= tw + 1,
        "counterexample": "" if sigma_dnf_val >= tw + 1 else f"f: {f}, pi: {pi}, sigma_dnf: {sigma_dnf_val}"
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        results.append(trial)

    sigma_dnf_vals = [r["metric_value"] for r in results]
    tw_plus_1_vals = [tw + 1 for tw in treewidths(results)]

    mean_sigma_dnf = sum(sigma_dnf_vals) / len(sigma_dnf_vals)
    std_deviation = (sum((x - mean_sigma_dnf)**2 for x in sigma_dnf_vals) / len(sigma_dnf_vals))**0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_sigma_dnf} std={std_deviation} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((seed for seed, r in zip(seeds, results) if not r["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f96783b5.py", line 135, in <module>
 trial = run_trial(seed)

~~~~~

File "/home/ludo/Scrivania/SEC/research/pvsnp\_sandbox/test\_f96783b5.py", line 115, in run\_trial

f = [[any(pi[i][j] == 'x' or pi[i][j] == term[j] for j in range(n)) for i in range(2\*n)] for term in terms]

~~~~~

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f96783b5.py", line 115, in <genexpr>

f = [[any(pi[i][j] == 'x' or pi[i][j] == term[j] for j in range(n)) for i in range(2*n)] for term in terms]

^^

NameError: cannot access free variable 'pi' where it is not associated with a value in enclosing scope

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a NameError before producing any data, so no instances were evaluated and neither the support nor falsification condition can be assessed. | next: Fix the scoping bug (pass pi explicitly into the comprehension or restructure the term-coverage computation) and rerun the 90-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	417704
2	propose	claude_max	opus	0	0	368130
3	preregistration	claude_max	opus	0	0	5587
4	novelty	claude_max	opus	0	0	3491
5	novelty	claude_max	opus	0	0	8732
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19180
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15876
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15189
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14966
10	judge	claude_max	opus	0	0	3882

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 872735 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 4}

(full prompt+response transcripts available in research/audit/614f26dd51ba.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/614f26dd51ba.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/614f26dd51ba.tar.gz (if generated)